Exercise Sheet 10

The Council of Guilds

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (venv active).
- One file per exercise in `my_work/`, run from the lesson folder: `python my_work/ex1_headcount.py`.
- Every exercise starts with the **opener** (type it, don't paste it). Exercises 3, 4 and 6 also need the `hourly` column, **unrounded**:

```
import pandas as pd

pd.set_option("display.width", 120)
df = pd.read_csv("data/thornmere_payroll.csv")
df["hourly"] = df["annual_salary"] / 52 / df["weekly_hours"]
```

- Printed tables must match exactly, including the row order and the little `Name:` / `dtype:` lines pandas adds under a Series.
- Hand in: commit and push to `lesson-10`, then open a pull request. Tell Lars which PivotTable you'll never build by hand again.

Exercise 1 Headcount

5 min · `ex1_headcount.py`

> **Raven from Lady Siv (Rewards):** *"Headcount by guild, by town, and the gender split as a share. It's a COUNTIF, I know, but I have eleven of them and one is wrong."*

(a) Print `df["department"].value_counts()`. Empty line. The same for location. Empty line.

(b) The gender split as shares: `value_counts(normalize=True)`, rounded to 3 dp. Empty line.

(c) How many different job titles there are, and the three most common.

EXPECTED OUTPUT

```
department
Forge         60
Guard         35
Harbour       25
Clockworks    20
Treasury      18
Library       15
Apothecary    15
Kitchens      12
Name: count, dtype: int64

location
Thornmere Keep    93
Westmarch         67
Harbourtown       40
Name: count, dtype: int64

gender
M    0.585
F    0.415
Name: proportion, dtype: float64
```

```
Job titles: 6
job_title
Journeyman    43
Craftsman     39
Apprentice    38
Name: count, dtype: int64
```

Hint: `value_counts()` is a PivotTable with one row field and "Count" in the values. Sorted biggest first, always.

Exercise 2 The PivotTable

10 min · `ex2_pivot_by_department.py`

> **Decree from Queen Aldara:** *"Average salary by guild. And the count, median and max while you're there. And which guild costs the Treasury most. One table, please, not four."*

After the opener: `by_dept = df.groupby("department")["annual_salary"]`.

(a) Print `by_dept.mean().round(0)`. Empty line.

(b) Print `by_dept.agg(["count", "mean", "median", "max"]).round(0)`. Empty line.

(c) Print the three biggest pay bills: `.sum()`, sorted largest first, `.head(3)`. Empty line.

(d) Print which guild has the biggest pay bill and which has the highest average (`.idxmax()`).

EXPECTED OUTPUT

```
department
Apothecary    33017.0
Clockworks    40415.0
Forge         29148.0
Guard         32264.0
Harbour       30844.0
Kitchens      33129.0
Library       30910.0
Treasury      32547.0
Name: annual_salary, dtype: float64

            count     mean   median    max
department
Apothecary     15  33017.0  31300.0  50150
Clockworks     20  40415.0  39925.0  60800
Forge          60  29148.0  28650.0  50500
Guard          35  32264.0  28200.0  61750
Harbour        25  30844.0  28950.0  47450
Kitchens       12  33129.0  31250.0  63650
Library        15  30910.0  29000.0  51600
Treasury       18  32547.0  29275.0  68750

department
Forge         1748900
Guard         1129250
Clockworks     808300
Name: annual_salary, dtype: int64

Biggest pay bill: Forge
Highest average: Clockworks
```

Hint: `groupby("x")["y"]` says "row field x, values y". What comes after it (`.mean()`, `.sum()`, `.agg([...])`) is the "Summarize values by" box.

Exercise 3 Two row fields

10 min · `ex3_two_keys.py`

> **Raven from Mistress Pell (Paymaster):** *"The gap by guild. Mean hourly rate for women and men, side by side, and the gap in percent, worst first. The Queen's words: 'worst first'."*

Opener plus the unrounded `hourly` column.

(a) Print the mean hourly rate by gender, 2 dp. Empty line.

(b) `both = df.groupby(["department", "gender"])["hourly"].mean()`. Print the first six rows, rounded to 2 dp. Empty line.

(c) `table = both.unstack().round(2)` turns the gender level into columns. Print it. Empty line.

(d) Add `gap_pct` = (M − F) / M × 100, rounded to 1 dp. Print the table sorted by gap, largest first.

EXPECTED OUTPUT

```
gender
F    14.95
M    18.62
Name: hourly, dtype: float64

department  gender
Apothecary  F         17.23
            M         18.91
Clockworks  F         17.05
            M         23.86
Forge       F         14.03
            M         15.88
Name: hourly, dtype: float64

gender          F      M
department
Apothecary  17.23  18.91
Clockworks  17.05  23.86
Forge       14.03  15.88
Guard       14.52  19.05
Harbour     11.56  18.07
Kitchens    14.17  25.81
Library     15.16  19.10
Treasury    16.72  19.81

gender          F      M  gap_pct
department
Kitchens    14.17  25.81     45.1
Harbour     11.56  18.07     36.0
Clockworks  17.05  23.86     28.5
Guard       14.52  19.05     23.8
Library     15.16  19.10     20.6
Treasury    16.72  19.81     15.6
Forge       14.03  15.88     11.6
Apothecary  17.23  18.91      8.9
```

Hint: Two keys need a **list**: `groupby(["a", "b"])`. And yes, that's the Kitchens at the top. The Head Cook has been invited to explain.

Exercise 4 pivot_table, the real one

10 min · `ex4_pivot_table.py`

> **Decree from Queen Aldara:** *"Can it look like an actual PivotTable? Guilds down, gender across, grand totals? I have been on the throne a long time."*

Opener plus the unrounded `hourly` column.

(a) `df.pivot_table(index="department", columns="gender", values="hourly", aggfunc="mean", margins=True)`, rounded to 2 dp. Empty line.

(b) A headcount table: same shape, `values="employee_id"`, `aggfunc="count"`, `fill_value=0`. Empty line.

(c) By town: `index="location"`, `aggfunc=["mean", "count"]` on hourly, rounded to 2 dp.

EXPECTED OUTPUT

```
gender          F      M    All
department
Apothecary  17.23  18.91  18.01
Clockworks  17.05  23.86  21.48
Forge       14.03  15.88  15.26
Guard       14.52  19.05  17.24
Harbour     11.56  18.07  16.51
Kitchens    14.17  25.81  18.05
Library     15.16  19.10  16.21
Treasury    16.72  19.81  18.26
All         14.95  18.62  17.10

gender       F   M
department
Apothecary   8   7
Clockworks   7  13
Forge       20  40
Guard       14  21
Harbour      6  19
Kitchens     8   4
Library     11   4
Treasury     9   9

                mean        count
gender             F      M     F   M
location
Harbourtown    13.98  17.76    20  20
Thornmere Keep 15.15  18.57    37  56
Westmarch      15.42  19.12    26  41
```

Hint: `margins=True` is the "Grand Total" row and column. `index` = Rows box, `columns` = Columns box, `values` + `aggfunc` = Values box.

Exercise 5 XLOOKUP

10 min · `ex5_xlookup.py`

> **Raven from Orrin of the Clockworks:** *"The Council of Guilds keeps its own list: guild, vault, guildmaster, budget. It's in `data/guilds.csv`. Mistress Pell XLOOKUPs it into the Roll every moon. That's 200 XLOOKUPs. One guild is missing from the Council's list, as usual."*

Opener, then `guilds = pd.read_csv("data/guilds.csv")`.

(a) Print `guilds`. Empty line.

(b) `merged = df.merge(guilds, on="department", how="left")`. Print its shape and the first three rows of first_name, department, vault, guildmaster. Empty line.

(c) How many people have no vault (`.isna().sum()`)? Which guild are they in (`sorted(... .unique())`)? Fill the gaps with `"VLT-999"` using `.fillna()` and print the count again. Empty line.

(d) Budget check: pay bill per guild (`groupby ... .sum().reset_index()`), merged with just `guilds[["department", "budget"]]` using `how="inner"`. Add `headroom` = budget − pay bill and print sorted by headroom, worst first.

EXPECTED OUTPUT

```
   department    vault       guildmaster   budget
0       Forge  VLT-100  Garrick Ironfoot  1750000
1     Harbour  VLT-110   Piran Saltmarsh   700000
2       Guard  VLT-200   Ulric Hawkridge  1150000
3    Treasury  VLT-300   Thea Goldenhand   540000
4    Kitchens  VLT-310       Hild Tallow   400000
5  Clockworks  VLT-400  Orrin Quickwater   780000
6     Library  VLT-500      Merewyn Pike   500000

(200, 15)
  first_name department    vault      guildmaster
0        Nye      Guard  VLT-200  Ulric Hawkridge
1       Hild    Library  VLT-500     Merewyn Pike
2      Maren      Guard  VLT-200  Ulric Hawkridge

15 people without a vault
['Apothecary']
0 after fillna

   department  annual_salary   budget  headroom
3     Harbour         771100   700000    -71100
6    Treasury         585850   540000    -45850
0  Clockworks         808300   780000    -28300
1       Forge        1748900  1750000      1100
4    Kitchens         397550   400000      2450
2       Guard        1129250  1150000     20750
5     Library         463650   500000     36350
```

Hint: `how="left"` keeps everyone from the left table, NaN where there's no match (XLOOKUP's "#N/A"). `how="inner"` keeps only matches, so the Apothecary silently disappears. Count your rows after every merge.

Exercise 6 Pay quartiles

10 min · `ex6_quartiles.py`

> **Decree from Queen Aldara:** *"The quartiles again, but as a proper table this time: four rows, women and men, counts and percentages. Last time it was five lines of text and the Privy Council asked for 'a grid'."*

Opener plus the unrounded `hourly` column, then `labels = ["Lower", "Lower middle", "Upper middle", "Upper"]`.

(a) `df["quartile"] = pd.qcut(df["hourly"], 4, labels=labels)`. Print `value_counts().sort_index()`. Empty line.

(b) `pd.crosstab(df["quartile"], df["gender"])`: counts. Empty line.

(c) The same with `normalize="index"`, times 100, rounded to 1 dp: the percentage of each quartile that is women / men. Empty line.

(d) The hourly range of each quartile: `groupby("quartile", observed=True)["hourly"].agg(["min", "max"])`, 2 dp.

EXPECTED OUTPUT

```
quartile
Lower           50
Lower middle    50
Upper middle    50
Upper           50
Name: count, dtype: int64

gender          F   M
quartile
Lower          36  14
Lower middle   23  27
Upper middle   15  35
Upper           9  41

gender            F     M
quartile
Lower          72.0  28.0
Lower middle   46.0  54.0
Upper middle   30.0  70.0
Upper          18.0  82.0

                 min    max
quartile
Lower          10.26  13.21
Lower middle   13.23  15.38
Upper middle   15.46  19.83
Upper          19.95  35.26
```

Hint: `qcut` cuts into equal-sized groups (50 people each); `cut` would cut into equal-width ranges. The Accord of Equal Coin wants `qcut`. Compare with lesson 08: same 72 % and 18 %.

Exercise 7 The bonus gap STRETCH

15 min · `ex7_bonus.py`

> **Raven from Lady Siv:** *"The Accord also wants the bonus gap: what share of women and men got a bonus, and the gap in the bonus amounts among those who did. I did it once, by candlelight. It took a day. Biscuit ate the parchment."*

Opener, then `df["got_bonus"] = df["bonus"] > 0`.

(a) Share receiving a bonus by gender: `groupby("gender")["got_bonus"].mean() * 100`, 1 dp. Empty line.

(b) `paid = df[df["got_bonus"]]`. Mean and median bonus by gender (`.agg(["mean", "median"])`), 2 dp. Empty line.

(c) Mean and median bonus gap, as a percentage of men's figure, 1 dp. Pick single cells out of a table with `table.loc["M", "mean"]`. Empty line.

(d) A pivot of mean bonus by guild and gender (from `paid`), with a `gap_pct` column, rounded to 1 dp and sorted largest gap first.

EXPECTED OUTPUT

```
gender
F    45.8
M    66.7
Name: got_bonus, dtype: float64
```

```
           mean  median
gender
F       1614.21  1235.0
M       2090.13  1785.0

Mean bonus gap: 22.8%
Median bonus gap: 30.8%

gender           F       M  gap_pct
department
Harbour      623.3  2573.6     75.8
Clockworks  1192.0  3053.8     61.0
Kitchens     780.0  1483.3     47.4
Apothecary  1936.7  2135.0      9.3
Guard       1874.0  1791.3     -4.6
Forge       2033.3  1865.8     -9.0
Library     1595.0  1350.0    -18.1
Treasury    2437.5  1807.5    -34.9
```

Hint: A negative gap means women's average bonus was higher in that guild. Four of eight. The Queen will ask why the Harbour is 75.8 %. (Nineteen men, six women, and the harbourmaster's bonus.)

Bug hunt Pages from the Grimoire, page 10

15 min · `bughunt/gap_by_dept_bug.py`

Recovered from Dalvric's locked "Dalvric working" tab, where it was pasted into a cell as text. It's the gap by guild with each guild's vault, for the Privy Council. **Three** bugs: one crashes (the error mentions a "level name"; think about how groupby takes two keys), one quietly loses fifteen people, one gets every percentage slightly wrong (compare with Exercise 3). One commit per bug, each message explaining what was wrong and why.

EXPECTED OUTPUT ONCE FIXED

```
People in report: 200
gender           F      M  gap_pct    vault
department
Kitchens     14.17  25.81     45.1  VLT-310
Harbour      11.56  18.07     36.0  VLT-110
Clockworks   17.05  23.86     28.5  VLT-400
Guard        14.52  19.05     23.8  VLT-200
Library      15.16  19.10     20.6  VLT-500
Treasury     16.72  19.81     15.6  VLT-300
Forge        14.03  15.88     11.6  VLT-100
Apothecary   17.23  18.91      8.9      NaN
```

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.